



"if를 통해 경기의 각팀이 넣은 골의 숫자를 비교하여 승패를 결정하고 각팀 최우수 선수의 활약 포인트를 비교하여 해당 경기의 mvp선수를 결정하고 if-elif-else 조건문을 통해 팀의 명성 합을 구간을 설정하여 일정 구간 이하 일때 이상일때 사이일때에 따라, 관중의 수도한 관중의 수가 어떤 구간에 있는지에 따...

◆ 제시해주신 과제 내용은 축구 시뮬레이션이나 게임 로직에서 아주 핵심적인 부분들이네요! 하지만 말씀하신 대로 모든 조건을 if-elif-else 로만 해결하려고 하면 코드가 소위 ***스파게티***처럼 꼬여서 나중에 수정하기도 힘들고 읽기도 어려워집니다.

이를 깔끔하게 해결할 수 있는 3가지 전략을 제안해 드릴게요.

1. 사전(Dictionary)을 활용한 점수/구간 관리

if 문을 줄이는 가장 좋은 방법은 데이터를 구조화하는 것입니다. 명성 합이나 관중 수에 따른 보상 배율을 딕셔너리에 저장해두면 조건문 없이도 값을 가져올 수 있습니다.

```
Python

# 예: 명성 구간에 따른 보상 가중치
reputation_bonus = {
    "low": 1.0,
    "medium": 1.5,
    "high": 2.0
}
```

① Gemini 프롬프트 입력

+ 🛠 도구 빠른 모델 🗲

이번 과제에서는 조건문 사용까지밖에 못하는데

◆ 아하, 과제 제한 사항이 **조건문(if-elif-else)만 사용 가능**한 거였군요! 그렇다면 제가 방금 보여드린 딕셔너리나 고급 함수 기법보다는, **구조를 잘 짠 조건문**으로 접근하는 게 정답입니다.

코드의 복잡성을 줄이는 핵심은 **조건문을 겹치지 않게(Flat하게) 배치하는 것**입니다. 모든 조건을 하나의 거대한 `if` 문 안에 넣으려 하지 말고, 단계별로 변수값을 업데이트하는 방식을 추천해요.

🌸 조건문만 사용한 깔끔한 코드 구조

Python

```
# 1. 기초 데이터 입력
team_a_score = 3
team_b_score = 2

team_a_mvp_point = 85
team_b_mvp_point = 90

total_prestige = 1200 # 팀 명성 합
spectators = 45000    # 관중 수
match_type = "final"  # 경기 중요도

# --- 단계 1: 승패 및 MVP 결정 ---
if team_a_score > team_b_score:
    winner = "A팀"
elif team_a_score < team_b_score:
    winner = "B팀"
```

① Gemini 프롬프트 입력

+ 🛠 도구

빠른 모델 ▾ 🗣

명지대학교 채팅은 Gemini 모델을 개선하는 데 사용되지 않습니다. Gemini는 AI이며 인물 등에 관한 정보 제공 시 실수를 할 수 있습니다. [개인 정보 보호 및 Gemini](#)

```
Python
-----

# 3) 경기 중요도 설정
if match_type == "final":
    reward = reward * 2.0
elif match_type == "semi-final":
    reward = reward * 1.5
# else는 변화가 없으면 굳이 안 써도 됩니다.

# --- 단계 3: 결과 출력 ---
print(f"경기 결과: {winner}")
print(f"MVP: {mvp}")
print(f"최종 지급 보상: {int(reward)}원")
```

🌟 왜 이 방식이 좋은가요?

- 1. 가독성: if 안에 if 를 넣는(중첩) 방식은 눈이 아프지만, 위에서 아래로 한 단계씩 계산하는 방식은 읽기가 훨씬 편합니다.
- 2. 수정 용이성: "관중 수에 따른 보상만 바꾸고 싶다"면 딱 그 if-elif 블록만 고치면 됩니다. 다른 코드에 영향을 주지 않아요.
- 3. 과제 의도 충족: if, elif, else 를 모두 적절하게 사용하면서도 논리적으로 완벽합니다.

이렇게 ***각 조건을 독립적인 if문 블록으로 나누기***만 해도 코드가 훨씬 정돈되어 보일 거예요. 이 구조대로 과제를 작성해 보시는 건 어떨까요?

① Gemini 프롬프트 입력

+ 🛠 도구

빠른 모델 ▾ 🎤